

MODULE 3: EVALUATION & METRICS

Contents

Classification based Model Evaluation Metrics	1
ROC Curve (Receiver Operating Characteristic Curve)	2
AUC (Area Under the ROC Curve)	3
Regression based Model Evaluation Metrics.....	4
Bias & Variance	4
Relationship Between Bias and Variance	5
Model Fitting in Machine Learning	5
Cross-Validation	6
Train–Test Split.....	6
K-Fold Cross-Validation.....	6
Stratified K-Fold Cross-Validation.....	7
Leave-One-Out Cross-Validation (LOO-CV).....	7
Ensemble Learning.....	10
Boosting — “Learn from your mistakes”	11
Bagging (Bootstrap Aggregating)	14
Random Forest.....	15
Introduction to Statistical Learning Theory	17
Risk and Error in Statistical Learning Theory	17

Classification based Model Evaluation Metrics

Confusion Matrix: A **confusion matrix** summarizes model predictions in a table:

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive (TP)	False Negative (FN)
Actual: Negative	False Positive (FP)	True Negative (TN)

$$\begin{aligned} \bullet \text{ Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \bullet \text{ Precision} &= \frac{TP}{TP + FP} \\ \bullet \text{ Recall (Sensitivity, TPR)} &= \frac{TP}{TP + FN} \end{aligned}$$

$$\begin{aligned} \bullet \text{ Specificity (TNR)} &= \frac{TN}{TN + FP} \\ \bullet \text{ F1-Score} &= 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})} \\ \bullet \text{ Balanced Accuracy} &= \frac{\text{Recall} + \text{Specificity}}{2} \end{aligned}$$

1. Accuracy

- Accuracy measures the **overall correctness** of a model.

- It tells us what fraction of total predictions were right — both positive and negative.
- It works well when **classes are balanced**, but can be misleading if one class dominates (e.g., 95% negative samples).
- **Meaning:** “Out of all predictions made, how many were correct?”

2. Precision

- Precision focuses only on **positive predictions** — it tells us how many of the predicted positives were actually true.
- A model with high precision makes **few false positive errors**.
- Used when **false alarms are costly**, such as spam detection or fraud alerts.
- **Meaning:** “When the model predicts positive, how often is it right?”

3. Recall / Sensitivity / True Positive Rate (TPR)

- These three terms — recall, sensitivity, and TPR — all mean the same thing.
- It measures how many of the **actual positive cases** the model correctly identified.
- Used when **missing a positive case is dangerous**, such as in medical tests.
- **Meaning:** “Out of all actual positives, how many did the model catch?”

4. Specificity (True Negative Rate, TNR)

- Specificity measures how many of the **actual negative cases** were correctly identified as negative.
- It’s the counterpart to recall but for negatives.
- Important in cases where **false positives** must be avoided.
- **Meaning:** “Out of all actual negatives, how many were correctly rejected?”

5. F1-Score

- The F1-Score combines **precision** and **recall** into a single measure.
- It is the **harmonic mean** of the two, giving a balanced view of a model’s performance when one metric alone might be misleading.
- Useful when dealing with **imbalanced datasets** or when both false positives and false negatives are important.
- **Meaning:** “A balanced score that considers both how accurate and how complete the positive predictions are.”

6. Balanced Accuracy

- Balanced accuracy averages **recall for both positive and negative classes**, giving a fair measure even if the dataset is imbalanced.
- Unlike normal accuracy, it doesn’t get biased by the majority class.
- **Meaning:** “Overall correctness, adjusted for class imbalance — gives equal weight to both classes.”

ROC Curve (Receiver Operating Characteristic Curve)

The **ROC curve** is a graphical way to evaluate how well a classification model can **distinguish between classes** (positive vs negative). It plots:

- **True Positive Rate (TPR or Recall)** on the **Y-axis**, and
- **False Positive Rate (FPR)** on the **X-axis**,

$$\text{FPR (or Fall-out)} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

$$\text{TPR (or Recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

for different threshold values of the model's output probability.

- In simple terms, each point on the ROC curve represents the model's performance at a specific decision threshold (for example, predicting "positive" if probability > 0.7 instead of 0.5).
- A model that can **perfectly distinguish** between classes will have a curve that rises quickly toward the top-left corner — meaning **high TPR** and **low FPR**.
- A random or poor classifier will produce a curve close to the **diagonal line**, which represents random guessing.
- **Meaning:** The ROC curve shows the trade-off between detecting positives and avoiding false alarms, across all possible decision thresholds.

AUC (Area Under the ROC Curve)

The **AUC** is a **single number summary** of the ROC curve — it measures the **entire area** under it. It represents the probability that the classifier will **rank a randomly chosen positive example higher than a randomly chosen negative example**.

- **AUC = 1.0:** Perfect classifier (always ranks correctly).
- **AUC = 0.5:** Random guessing (no discrimination ability).
- **AUC < 0.5:** Worse than random (model predictions are inverted).

Meaning: AUC measures how well the model separates the positive and negative classes, independent of the threshold chosen.

Why ROC–AUC is Important

- It's **threshold-independent** — evaluates model quality without fixing a specific cutoff (like 0.5).
- Useful for comparing multiple models' discriminative power.
- Works even with **imbalanced datasets**, unlike raw accuracy.

Regression based Model Evaluation Metrics

1. Mean Squared Error (MSE)

- MSE measures the **average squared difference** between actual and predicted values.
- A **lower MSE** means predictions are closer to actual outputs.
- Squaring amplifies the impact of large errors, making MSE sensitive to outliers.
- It is a **loss function** often used during model training (e.g., Linear Regression).
- **Intuition:** “How far off are predictions on average, penalizing big mistakes heavily?”

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2. Root Mean Squared Error (RMSE)

- RMSE is the **square root of MSE**, giving errors in the **same units** as the target variable.
- It's easier to interpret practically (e.g., if predicting house prices in lakhs, RMSE will also be in lakhs).
- Like MSE, it penalizes large errors more heavily.
- **Intuition:** “The typical magnitude of prediction error in the same scale as the actual data.”

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

3. Coefficient of Determination (R² or R-squared)

- R² measures how well the regression model **explains the variance** in the target variable.
- Value lies between **0 and 1**:
 - **R² = 1**: Perfect prediction (model explains all variance).
 - **R² = 0**: Model does no better than the mean.
 - **R² < 0**: Model performs worse than predicting the average.
- **Intuition:** “What fraction of the total variation in the output is explained by the model?”

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- y_i : actual values
- $\hat{y}_i$: predicted values
- $\bar{y}$: mean of actual values

Bias & Variance

Bias

- **Bias** measures how far a model's **predictions are from the true values** on average.
- It represents **error due to simplifying assumptions** made by the model.
- A **high bias** model is too simple and cannot capture the true underlying pattern. It underfits the data and cannot capture important patterns in the data, leading to poor performance on both training and testing sets.
- **Low bias** means when model is sufficiently complex and can capture the true relationships between input and output.
- **Example:** Using a linear model to fit curved data → consistently wrong predictions.
- **Intuition:** High bias = model is too rigid or simplistic → underfits the data.

Variance

- **Variance** measures how much the model's **predictions change** if trained on different samples of data.

- It represents **sensitivity to training data** — high variance models memorize instead of generalizing.
- A **high variance** model performs well on training data but poorly on unseen data. Model is too complex it overfits the data.
- Low variance means the model is general enough to perform consistently across both training and testing data.
- **Example:** A deep decision tree that fits every training point exactly → overfits.
- **Intuition:** High variance = model is too complex → overfits the data.

Relationship Between Bias and Variance

There is a **trade-off** between bias and variance:

- Reducing bias (by making model more complex) often **increases variance**.
- Reducing variance (by simplifying the model) often **increases bias**.

Mathematically, total prediction error can be expressed as:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Bias²:** Error from wrong assumptions.
- **Variance:** Error from sensitivity to data fluctuations.
- **Irreducible Error:** Noise inherent in the data (can't be reduced by any model).

Goal: Find a model that minimizes **both bias and variance** to achieve the lowest total error.

The **bias–variance trade-off** represents the balance between underfitting and overfitting:

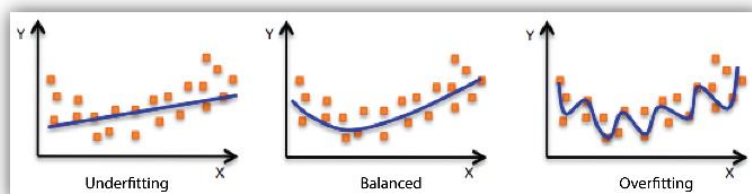
Model Complexity	Bias	Variance	Performance
Too Simple	High	Low	Underfitting
Optimal	Moderate	Moderate	Best Generalization
Too Complex	Low	High	Overfitting

Graphically, as model complexity increases:

- **Bias** ↓ decreases
- **Variance** ↑ increases
- **Total error** first decreases, then increases — minimum point is **optimal complexity**.

Model Fitting in Machine Learning

When training a model, the goal is to learn patterns from the training data that generalize well to unseen data. However, if the model is **too simple** or **too complex**, it can fail to generalize properly. This leads to two key issues: **Underfitting** and **Overfitting**.



1. Underfitting

- Occurs when the model is **too simple** to capture the underlying relationship in the data.
- The model performs **poorly on both training and test sets**.
- Common causes:
 - Too few features or parameters
 - High bias (oversimplified assumptions)
 - Insufficient training

- Example: Using a linear model for highly non-linear data.
- **Meaning:** “The model hasn’t learned enough — it fails even on training data.”

2. Overfitting

- Happens when the model is **too complex** and learns **noise or random fluctuations** in the training data instead of the true pattern.
- It performs **very well on training data** but **poorly on unseen (test) data**.
- Common causes:
 - Too many parameters
 - Too little data
 - Lack of regularization
- Example: A high-degree polynomial fitting a small dataset.
- **Meaning:** “The model learned too much — including irrelevant details.”

Cross-Validation

- Cross-validation is a **model evaluation and validation technique** used to assess how well a model will **generalize to unseen data**.
- Instead of relying on a single train–test split, it repeatedly splits the dataset in different ways to get a **more reliable estimate of model performance**.
- **Goal:** To prevent overfitting and ensure the model performs well on new data.

Train–Test Split

- The simplest form of validation.
- The dataset is divided into two parts:
 - **Training set** — used to fit the model.
 - **Testing set** — used to evaluate performance.
- Common split ratios: **70–30**, **80–20**, or **90–10**.
- Model is trained once and tested once.
- **Advantages:** Simple and fast.
- **Limitations:** Evaluation depends on how the data was split — may not represent overall performance accurately.

K-Fold Cross-Validation

- The dataset is divided into **K equal-sized folds (subsets)**.
- The model is trained on **K–1 folds** and tested on the **remaining fold**.
- This process is repeated **K times**, each time with a different fold as the test set.
- The final performance metric is the **average of all K test results**.

Example: If $K = 5 \rightarrow$ The data is split into 5 parts, and the model trains 5 times with a different part for testing each time.

Advantages:

- Uses all data points for both training and testing (in rotation).
- Provides a more stable and reliable performance estimate.
- Reduces variance due to random train–test splits.

Common choice: $K = 5$ or 10 .

Stratified K-Fold Cross-Validation

- Stratified K-Fold CV is a variant of K-Fold CV where the splitting process is stratified — that means each fold contains approximately the same percentage of samples of each target class as the complete dataset.
- When to Use: Always for classification tasks — especially with imbalanced classes. Examples:
 - Spam detection (95% non-spam, 5% spam)
 - Medical diagnosis (rare disease data)
 - For regression problems, standard K-Fold is fine (since target is continuous).
- Advantages
 - Keeps class proportions consistent across folds
 - Produces more stable and reliable validation scores
 - Reduces bias in model evaluation on imbalanced data

Leave-One-Out Cross-Validation (LOO-CV)

- Special case of K-Fold where $K = N$ (number of data points).
- Each sample is used once as the test set, and all others as training data.
- Very accurate but computationally expensive for large datasets.

When to Use Which

Method	Description	Pros	Cons	Use Case
Train–Test Split	One-time split	Fast, simple	High variance	Quick checks, large datasets
K-Fold CV	Repeated K splits	Reliable estimate	Slower	Small–medium datasets
LOOCV	$K = N$	Most accurate	Very slow	Tiny datasets

Example 1: Stratified K fold Cross Validation

Sample No.	1	2	3	4	5	6	7	8	9	10
Class	A	A	A	A	A	B	B	B	B	B

So total class ratio = **50% A : 50% B**

Stratified 5-Fold Cross-Validation Splits:

Each fold should have **1 sample from Class A** and **1 sample from Class B**.

Fold	Test Samples	Class Composition	Train Samples (Remaining 8)
Fold 1	[1, 6]	(A, B)	[2,3,4,5,7,8,9,10]
Fold 2	[2, 7]	(A, B)	[1,3,4,5,6,8,9,10]
Fold 3	[3, 8]	(A, B)	[1,2,4,5,6,7,9,10]
Fold 4	[4, 9]	(A, B)	[1,2,3,5,6,7,8,10]
Fold 5	[5, 10]	(A, B)	[1,2,3,4,6,7,8,9]

Example 2: Stratified K fold Cross Validation

We have 12 samples, belonging to 2 classes (A and B).

Sample No.	1	2	3	4	5	6	7	8	9	10	11	12
Class	A	A	A	A	A	A	B	B	B	B	B	B

So the class ratio is 6 A : 6 B (50–50).

We'll use 4 folds (K = 4).

Goal

Each fold should contain **equal class proportion** — i.e., Since total 12 samples → 4 folds ⇒ each fold has **3 samples total**. So we'll distribute like this:

- 2 folds have 2 A + 1 B
- 2 folds have 1 A + 2 B

Stratified 4-Fold Split (Balanced Distribution)

Fold	Test Samples	Class Composition	Train Samples
Fold 1	[1, 7, 8]	(1A, 2B)	[2,3,4,5,6,9,10,11,12]
Fold 2	[2, 3, 9]	(2A, 1B)	[1,4,5,6,7,8,10,11,12]
Fold 3	[4, 10, 11]	(1A, 2B)	[1,2,3,5,6,7,8,9,12]
Fold 4	[5, 6, 12]	(2A, 1B)	[1,2,3,4,7,8,9,10,11]

Example 3: Cross Validation vs Train-Test Split

You train a model using an 80–20 train–test split and get 95% accuracy on training data and 75% on test data.

Then you apply 5-fold cross-validation and obtain accuracies:

82%, 78%, 80%, 81%, 79%.

(a) Find the average cross-validation accuracy.

(b) What does the difference between test accuracy (75%) and CV accuracy tell you about model overfitting?

④ Arg cross-val. acc = $\frac{82+78+80+81+79}{5} = 80\%$

⑤ Interpretation.

Metric	Value	Interpretation
Training Accuracy	95	High, so fits good for training data
Testing Accuracy	75	Low very, so for new unknown data model fails. Overfitting.
Mean accuracy (5 fold) Cross Valid.	80	This is cross validation, higher than single test accuracy. More reliable as done on 5 splits

Conclusion

→ Overfitting & high variance. Perform good on known data but not on unseen one.

Example 4: Leave One Out Cross Validation

Q) 4 datapoints: $(1, 2), (2, 4), (3, 6), (4, 8)$. & a simple linear reg model $y = wx + b$. Use LOOCV to find MSE.
Assume Predicted ($\hat{y}$) for 4 itr is 2.2, 3.9, 6.1, 7.8.

Ans) Here for LOOCV, No of datapoints = N .
 $\therefore (N-1)$ training points & 1 testing point.

Iteration	Training Set	Test Point	$\hat{y}$	$y - \hat{y}$
1	$(1, 2), (2, 4), (3, 6)$	$(4, 8)$	7.8	0.2
2	$(1, 2), (2, 4), (4, 8)$	$(3, 6)$	6.1	-0.1
3	$(1, 2), (3, 6), (4, 8)$	$(2, 4)$	3.9	0.1
4	$(2, 4), (3, 6), (4, 8)$	$(1, 2)$	2.2	-0.2

$$\begin{aligned} \text{MSE} &= \frac{1}{N} \sum (y_i - \hat{y}_i)^2 = \frac{1}{4} (0.2^2 + 0.1^2 + 0.1^2 + 0.2^2) \\ &= \frac{1}{4} ((0.04 + 0.01) \times 2) = \frac{1}{4} (0.06) = 0.015 \end{aligned}$$

Example 5: Leave One Out Cross Validation

You have 4 data points:

$(1, 1), (2, 2), (3, 3), (4, 4)$.

A simple linear regression model $y = wx + b$ is trained using LOOCV, i.e., training on 3 points and testing on the remaining one each time.

Given predicted vs actual errors for each left-out sample are:

0.1, -0.2, 0.0, 0.1

Find the average MSE over all LOOCV iterations.

Ans) $\text{MSE} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$ here $N=4$

Error ($y_i - \hat{y}_i$)	$\sum (y_i - \hat{y}_i)^2$
+0.1	0.01
-0.2	0.04
0.0	0
+0.1	0.01

$\therefore \text{MSE} = \frac{1}{4} (0.06) = 0.015$

Conclusion: MSE is quite small, so model fits well & is with low bias & low variance.

Example 6: Bias-Variance from Cross Validation

You perform 10-fold cross-validation on two models:

Model	Mean Accuracy	Std. Deviation
A	94%	5%
B	92%	1%

Which model has higher bias? Which has higher variance? Explain briefly using CV results.

Ans) Bias $\rightarrow$ how diff model is from actual prediction
(how far model's pred. from true value)

Variance $\rightarrow$ how model prediction changes with diff data samples.

⑥ Bias $\rightarrow$ accuracy (High accuracy = low bias)
 $\therefore$ Model B is less accurate than A. B is more bias.

⑦ Variance $\rightarrow$ Sd. (More variance = more Sd)
 $\therefore$ A has more Sd. than B. A is more variance

⑧ Interpretation.

Model	Bias	Variance	Interpretation.
A	Low	High	Overfitting. Accurate on training data, but less accurate for unseen data.
B	High	Low	Underfitting. Less accurate for any data.

Ensemble Learning

- **Ensemble Learning** is a technique in Machine Learning where **multiple models (learners)** are trained and then **combined** to produce a stronger, more accurate overall model.
- Each model is called a **base learner** or **weak learner**, and the ensemble combines them to reduce **error, variance, or bias**.
- **Intuition:** "Two (or more) weak models together can make a strong one — like a group of average people making smarter decisions than one expert."
- **Why Ensembles Work:** Every model makes slightly different errors. When combined intelligently, their mistakes can **cancel out**, leading to:
 - **Higher accuracy**
 - **Lower variance**
 - **Better generalization**
- This is known as the **wisdom of crowds** effect.

- If individual models $h_1, h_2, \dots, h_k$ make predictions, then ensemble prediction is:

$$\hat{y} = \text{Combine}(h_1(x), h_2(x), \dots, h_k(x))$$

where **Combine** can be:

- **Mean** → regression
 - **Majority vote** → classification
 - **Weighted average** → boosting/stacking
- **Example Analogy:** Imagine 3 classifiers:
 - Model 1 predicts 80% correctly
 - Model 2 predicts 75% correctly
 - Model 3 predicts 78% correctly

If their **errors differ**, then combining their outputs (say, via majority vote) might achieve **85–90% accuracy** overall — that’s the power of ensembles.

Boosting — “Learn from your mistakes”

Definition

Boosting is an **ensemble technique** where models are trained **sequentially**, and each new model **tries to correct the errors** made by the previous ones.

All the models’ predictions are then **combined** (usually as a weighted sum) to make the final decision.

Goal: Convert several *weak learners* (models performing just slightly better than random guessing) into a **strong learner**.

Core Intuition

Think of it like a teacher helping a struggling student:

- The first test shows where the student went wrong.
- Next time, the teacher **focuses only on those mistakes**.
- Gradually, the student improves.

Boosting works the same way:

Every new model **focuses on data points the previous models misclassified**.

How Boosting Works

Step	Description
1. Initialize	Start with equal weights for all samples (so every sample is equally important).
2. Train 1st model	Fit a weak learner (e.g., small Decision Tree). Compute its error rate.
3. Update weights	Increase weights for misclassified samples → the next model focuses more on these.
4. Train next model	Fit a new weak learner using the updated weights.
5. Repeat	Continue for T iterations.
6. Combine models	Final output = weighted sum (or vote) of all learners.

❏ Example (Conceptual)

Suppose you have 100 samples. Your first weak learner (say a small tree) gets 80 right, 20 wrong. Boosting increases the **importance (weight)** of those 20 misclassified samples.

→ The next learner focuses more on them.

→ This cycle repeats, each model improving the previous one's mistakes.

Final prediction = weighted combination of all learners.

❏ Popular Boosting Algorithms

Algorithm	Description	Key Idea
AdaBoost (Adaptive Boosting)	Sequentially adjusts sample weights after each learner.	Focuses more on misclassified points.
Gradient Boosting	Sequentially fits new models to residual errors of previous models.	Uses gradient descent to minimize loss.
XGBoost / LightGBM / CatBoost	Advanced, optimized versions of Gradient Boosting.	Faster, regularized, handles large data efficiently.

❏ AdaBoost (Core Example)

- AdaBoost trains models **sequentially**, where each model **focuses more on the data points misclassified by the previous models**.
- It **adapts** the weight of each sample depending on how difficult it is to classify — hence “Adaptive” Boosting.
- Early models focus on all data.
- Later models **zoom in** on difficult cases.
- Each model gets a **voting weight** based on accuracy.
-

◆ Working Steps

Let's say we have training data $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ and weak learners $h_t(x)$ (often shallow decision trees).

1. Initialize weights:

Each data point has equal importance initially.

$$w_i = \frac{1}{N}$$

2. Train weak learner:

Fit a model $h_t(x)$ using the weighted dataset.

3. Calculate weighted error:

$$\epsilon_t = \frac{\sum_i w_i [h_t(x_i) \neq y_i]}{\sum_i w_i}$$

4. Compute model weight:

(Higher weight → more trusted learner)

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

5. Update sample weights:

Misclassified samples get higher weights.

$$w_i \leftarrow w_i \times e^{-\alpha_t y_i h_t(x_i)}$$

6. Normalize weights (sum = 1)

Then repeat for T rounds.

7. Final strong classifier:

Weighted combination of all weak learners:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

✅ Each learner focuses more on “hard” examples → sequential improvement.

▣ Gradient Boosting — the modern version

- Instead of changing weights, each model predicts the **residual error** (difference between actual and predicted values). Then the next model learns to correct that residual.
- Think of it as **gradient descent in function space** — each weak learner reduces the loss step by step.

◆ Working Steps

Suppose the model aims to minimize a **loss function** $L(y, F(x))$.

1. Initialize model:

(e.g., the mean for regression)

$$F_0(x) = \arg \min_c \sum_i L(y_i, c)$$

2. For each iteration $m = 1, \dots, M$:

- Compute **residuals** (negative gradient):

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

- Fit a new weak learner $h_m(x)$ to predict these residuals.

- Find optimal multiplier γ_m :
$$\gamma_m = \arg \min_{\gamma} \sum_i L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$$

- Update model:
$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

✓ Each new model tries to **correct the errors (residuals)** of the previous ensemble.

8 Advantages

- ✓ Reduces bias → makes weak models strong
 - ✓ Works well with simple base learners (like shallow trees)
 - ✓ Often achieves **state-of-the-art accuracy** (especially XGBoost)
-

9 Disadvantages

- ✗ Computationally expensive (sequential training)
- ✗ Sensitive to noise and outliers (especially AdaBoost)
- ✗ Harder to tune (many hyperparameters)

Bagging (Bootstrap Aggregating)

❏ Definition

Bagging (short for **Bootstrap Aggregating**) is an **ensemble learning technique** that builds **multiple independent models** on **different random subsets** of the dataset and then **combines their predictions** (by averaging or voting) to make the final output.

❏ Intuition

- Imagine you have one weak model that performs okay, but not great — it overfits slightly. Now, if you train **many such models on slightly different data samples**, each one will learn **slightly different patterns**.
- When you **combine** (average or vote) their predictions, the random errors **cancel out**, and the overall model becomes **more stable and accurate**.
- ✅ **Goal:** Reduce **variance** without increasing bias.

❏ The Process

Step	Action	Description
1	Bootstrapping (Sampling)	Create k new datasets by randomly sampling from the original data with replacement . Each bootstrap sample has the same size as the original dataset.
2	Model Training	Train a separate model on each bootstrap sample (e.g., decision tree).
3	Aggregation (Voting/Averaging)	Combine all model predictions: - For classification → majority vote - For regression → average the predictions.

❏ Example

Let's say you have 1000 samples.

- You create **5 bootstrap samples**, each randomly drawn (with replacement) from those 1000 points.
- Train **5 models** (M_1, M_2, M_3, M_4, M_5) — each sees a slightly different view of the data.
- During prediction, combine all 5 outputs:

Regression:

$$\hat{y} = \frac{1}{5} \sum_{i=1}^5 h_i(x)$$

Classification:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), h_3(x), h_4(x), h_5(x)\}$$

❏ Key Idea — Bootstrap Sampling

Sampling with replacement means:

- Each bootstrap dataset contains duplicates.
- Some original data points are **not included** (~36.8% expected to be missing).
- This randomness increases model diversity → better generalization.

(Each sample contains ~63.2% unique points from the original.)

❏ Why Bagging Works

- Individual models have **high variance**, but **low bias** (like decision trees).
- Averaging them **reduces variance** (fluctuations due to random noise) without increasing bias much.

Hence: Bagging is best for **unstable, high-variance models** (like Decision Trees).

❏ Example Algorithm — Random Forest

Random Forest = Bagging + Random Feature Selection

- Each tree trained on a **bootstrap sample**
 - At each split, only a **random subset of features** is considered
→ Increases diversity → Reduces correlation between trees → Stronger ensemble.
-

❏ Mathematical View

If individual models have errors ϵ_i with mean 0 and variance σ^2 , then the average of N such models reduces variance to:

$$\text{Var(Ensemble)} = \frac{\sigma^2}{N}$$

(Assuming model errors are uncorrelated.)

- ✓ So, more models → lower variance → smoother and more reliable predictions.
-

❏ Advantages

- ✓ Reduces variance → less overfitting
 - ✓ Works well with unstable learners (Decision Trees)
 - ✓ Simple, parallelizable
 - ✓ Robust against noise and outliers
-

❏ Disadvantages

- ✗ Increases computation cost (multiple models)
 - ✗ Doesn't reduce bias (if base model is underfitting, bagging can't fix it)
 - ✗ Less interpretable
-

Random Forest

- **Random Forest** is an **ensemble of Decision Trees**, trained using **Bagging (Bootstrap Aggregating)** and **Random Feature Selection**, to improve accuracy, stability, and generalization.
 - Each tree gives a prediction, and the forest **aggregates** (averages or votes) the results.
 - **Idea:** Many weak, overfitting trees → when combined → form a strong, robust predictor.
-

❏ Why Random?

Because Random Forest introduces **two levels of randomness**:

1. **Random sampling of data** (bootstrap sampling).
2. **Random sampling of features** at each tree split.

This randomness ensures trees are **decorrelated**, so their errors don't overlap much — leading to lower variance and better generalization.

❏ Intuition

Each tree is a bit “noisy” and different because:

- It saw a slightly different subset of data.
- It considered only some features at each node.

Individually they might be weak, but together they’re strong and stable — because their **errors average out**.

✓ Think of Random Forest as “wisdom of trees.”

4 Prediction

Regression:

$$\hat{y} = \frac{1}{5} \sum_{i=1}^5 h_i(x)$$

Classification:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), h_3(x), h_4(x), h_5(x)\}$$

4 Feature Randomness

At each split in a tree:

- Instead of checking *all features*, Random Forest checks only a **random subset (m features)**.
- For classification → typically $m = \sqrt{d}$
- For regression → typically $m = d/3$
(where d = total number of features)

This reduces **correlation** between trees → better ensemble performance.

5 Mathematical Intuition

Let:

- Each tree have error ϵ
- Correlation between trees = ρ

Then ensemble variance $\approx$

$$\text{Var}_{\text{forest}} = \rho\sigma^2 + \frac{(1 - \rho)}{N}\sigma^2$$

→ If ρ (correlation) is low, and N is large → overall variance is small.

→ That’s why Random Forest works: it **reduces variance** by making trees less correlated.

6 Advantages

- ✓ Handles both classification & regression
 - ✓ Resistant to overfitting (thanks to bagging)
 - ✓ Works well with large data & high-dimensional features
 - ✓ Provides **feature importance**
 - ✓ Stable & high accuracy with minimal tuning
-

7 Disadvantages

- ✗ Slower to train for huge datasets
- ✗ Harder to interpret (many trees)
- ✗ Can still overfit if trees are too deep or too many features are correlated

Introduction to Statistical Learning Theory

Statistical Learning Theory provides the *mathematical foundation* of machine learning — it studies how an algorithm can learn a general rule (function) from a finite amount of data, and how well it will perform on unseen data.

It formalizes **learning as a statistical estimation problem** — balancing between **fitting data** and **generalizing** to new examples.

Risk and Error in Statistical Learning Theory

□ The Core Idea

In Statistical Learning Theory, the **goal of machine learning** is to find a function $f(x)$ that predicts output y from input x **with the smallest possible error on unseen data**.

This error (or loss) can be measured in two ways:

1. **Empirical Risk** → measured on **training data**
2. **Expected Risk** → measured on the **true data distribution**

The **ultimate goal** of any ML algorithm is to **minimize the Expected Risk**, but since we don't know the true data distribution, we instead minimize the **Empirical Risk**.

⌚ Expected Risk (True Risk)

Expected Risk represents the **true average loss** the model would have if we could test it on **all possible data points** from the real-world distribution $P(x, y)$.

$$R(f) = \mathbb{E}_{(x,y) \sim P}[L(y, f(x))] = \int L(y, f(x)) dP(x, y)$$

Where:

- $L(y, f(x))$ = loss function (e.g., squared error, 0–1 loss)
- $P(x, y)$ = true, unknown joint probability distribution

✓ This is what we *really want to minimize* — the **Expected (True) Risk**.

✗ But in reality, we don't have access to all data or $P(x, y)$.

⌚ Empirical Risk

Since we only have a finite sample dataset

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

we approximate the Expected Risk using the average loss over training samples:

$$R_{emp}(f) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))$$

✓ This is what we can *actually compute and minimize* using algorithms (e.g., gradient descent). It's the model's **training error**.

4 Relationship Between the Two

Concept	Symbol	Meaning
Expected Risk (True Risk)	$R(f)$	True generalization error on all possible data
Empirical Risk	$R_{emp}(f)$	Observed error on training dataset
Goal	Minimize $R(f)$	Approximate it by minimizing $R_{emp}(f)$

Ideally: $R_{emp}(f) \approx R(f)$ but in reality: $R_{emp}(f) < R(f)$ if the model **overfits**.

5 The Learning Challenge

5 The Learning Challenge

Because $P(x, y)$ (true data distribution) is unknown,
we cannot compute $R(f)$ directly — we only estimate it through $R_{emp}(f)$.

So, the **main question** in SLT is:

How close is $R_{emp}(f)$ to $R(f)$?
and under what conditions does minimizing training error lead to low true error?

This leads to **Generalization Bounds** (covered later): $R(f) \leq R_{emp}(f) + \text{Complexity penalty}$
which connects **training error** and **true error** mathematically.

6 Key Intuition

- **Empirical Risk** = what the model *thinks* it's doing well on (training data).
- **Expected Risk** = what the model *actually* does well on (unseen data).
- The **gap** between them measures **generalization ability**.

A good ML model keeps this gap **small**.